

操作系统 - 第六周星期三：内存自映射

课程导入

上次课我们讲了缺页处理和页面置换算法。今天我们讲一个实际实现中很关键的问题：**页表本身放在哪里？操作系统填写页表时，应该用虚拟地址还是物理地址填写？**

这个问题看起来绕，但实际是操作系统启动过程中必须解决的"先有鸡还是先有蛋"问题。

第一节：多级页表回顾

我们先快速回顾一下分页和多级页表的基础知识：

分页基本思想

分页的核心思想就是**破除内存分配的连续性假设**：

- 把虚拟地址空间按页大小划分
- 把物理内存按相同大小划分为页框
- 页表记录**虚拟页** → **物理页框**的映射关系

对于32位地址空间，页大小4KB，那么：

- 页内偏移占 12 位（因为 $2^{12} = 4096 = 4KB$ ）
- 剩下 20 位是页号 → 一共 $2^{20} = 1M$ 个虚拟页

如果用**一级页表**：

- 需要 1M 个页表项，每个页表项4字节 → 总共 **4MB** 连续物理内存存放页表
- 问题：4MB连续内存不好分配，而且大部分进程根本用不到这么多页，浪费空间

二级页表解决方案

多级页表就是对页表本身再分页，破除**页表存储的连续性假设**：

32位二级页表地址划分：

段	位数	作用
页目录索引	10位	定位页目录项
页表索引	10位	定位页表项
页内偏移	12位	页内偏移

总共：(10+10+12 = 32) 位，正好32位。

这样设计的好处：

- 页目录占 1024 个页目录项 (2^{10}) → 一页就放下了

- 只有用到的页表才分配物理页框，不用的不分配 → 节省内存
- 页表本身也可以离散存放，不需要连续

第二节："先有鸡还是先有蛋"问题

现在问题来了：**页表本身也要存在内存里，操作系统要填写页表，填写的时候要访问内存，访问内存就需要地址翻译，地址翻译又需要页表。**

更具体地说：

操作系统要填写页表项，填写页表项这个操作本身，就是一条写内存指令，指令里的地址是虚拟地址还是物理地址？

如果你说**用虚拟地址填写**：

- 那虚拟地址要翻译成物理地址，必须已经有页表映射
- 但你现在正在填写页表，页表还没填好 → 鸡生蛋蛋生鸡问题

如果你说**用物理地址填写**：

- 那开了分页之后，CPU所有访存都是虚拟地址，你无法直接使用物理地址
- X86开了分页后，所有地址都要经过页表翻译 → 物理地址不能直接用

这就是悖论。怎么解决？

第三节：MIPS架构的解决方案——分段直接映射

不同硬件架构有不同的解决方法。我们先看MIPS架构（你们实验用的就是MIPS）。

MIPS地址空间划分

MIPS32位（4GB）虚拟地址空间，硬件**人为划分**成几段：

地址范围	名称	映射方式	用途
0x00000000 ~ 0x7FFFFFFF	kuseg	页表映射	用户空间
0x80000000 ~ 0x9FFFFFFF	kseg0	直接映射	内核代码、数据
0xA0000000 ~ 0xBFFFFFFF	kseg1	直接映射	BIOS/启动 ROM
0xC0000000 ~ 0xFFFFFFFF	kseg2	页表映射	内核动态分配

直接映射是什么意思？

- 硬件直接把虚拟地址的高位置零，得到物理地址
- 不需要走页表 → 虚拟地址 = 物理地址 + 偏移，硬件自动转换

所以，MIPS内核代码本身放在kseg0/kseg1，这些地址硬件直接映射，**不需要页表就能访问物理内存。**

解决悖论

内核启动的时候：

- 内核代码本身在kseg0，直接访问物理内存，不需要页表
- 内核填写用户进程页表的时候，内核自己用的是**直接映射的虚拟地址**，其实就是物理地址
- 所以填写页表没问题，先填好用户页表，用户进程就能跑了

悖论解决了！因为有一块地址不需要页表就能用，内核用这块地址来填写其他页表。

你们实验中内核入口地址 **80040000** 就是kseg0的地址，正好印证了这一点。

第四节：X86架构的解决方案——内存自映射

如果硬件不提供直接映射区域怎么办？比如X86，开了分页之后所有地址都要走页表。这时候就要用**内存自映射**技术解决。

核心思想

把内核页表本身，也映射到内核虚拟地址空间的一个固定位置。这样内核就能用虚拟地址访问页表了。

具体来说：

整个4GB虚拟地址空间中，内核拿出一块连续的4MB虚拟地址空间，专门用来放整个页表。这一块就是**自映射区域**。

自映射原理（二级页表为例）

我们来拆解一下：

1. **一级页表（页目录）**：有 1024 个页目录项（PDE），每个PDE 4字节 → 总共 $1024 \times 4B = 4KB$ → 正好一页
2. 每个PDE指向一个二级页表（页表页），每个二级页表也有 1024 个PTE，正好一页
3. **自映射怎么做：**
 - 页目录自己**也是一个页表页**
 - 我们在页目录中，专门留出一个PDE，这个PDE**指向页目录自己所在的物理页框**
 - 这样，通过这个PDE，就能访问整个页目录所有PDE → 也就访问了所有页表页
4. 结果：整个四级页表（包括页目录和所有页表页）都被映射到了这 4MB 虚拟地址空间中
 - 内核只要拿到这个虚拟地址区域，就能访问所有页表项
 - 不需要物理地址，直接用虚拟地址读写页表

这个就叫**自映射**——页表自己映射自己。

为什么这能解决悖论？

一旦你完成了自映射的设置：

- 整个页表已经映射到虚拟地址空间了

- 内核以后要修改任何页表项，直接用虚拟地址访问就行
- 虚拟地址翻译能正常工作，因为映射已经建立好了

那最初怎么建立自映射？

- 启动阶段：先开启分页前，内核在低地址实模式下建立最初的页目录和页表，这时候用物理地址没问题
- 建立好之后，把那个特殊的PDE指向页目录自己的物理页框
- 然后开启分页，跳转到虚拟地址执行 → 自映射就完成了
- 之后所有操作都用虚拟地址，悖论解决

第五节：为什么要自映射？总结一下好处

1. **解决悖论**：内核可以用虚拟地址访问页表，不需要特殊的"直接访问物理内存"支持
2. **统一地址空间**：所有页表都在一个连续虚拟地址区间，内核访问方便
3. **内核可以管理所有页表**：不需要为页表单独留一块物理地址空间，完全虚拟地址空间管理

第六节：单位换算基本功（考试/实验都要用）

最后强调一下基本功：地址、大小、单位换算一定要熟练。

常见换算

名称	大小	(2 ⁿ)	说明
1KB	1024 Bytes	(2 ^{10})	
1MB	1024 KB	(2 ^{20})	
1GB	1024 MB	(^{30})	
4KB	4096 Bytes	(2 ^{12})	常见页大小
4MB		(2 ^{22})	自映射区域大小

地址对齐判断

- 4KB对齐 → 地址低12位是0 → 十六进制地址末尾是 000
- 如果地址末尾是 800，那它不是4KB对齐，因为 (800_{16} = 2048_{10})，低12位不是全0

计算题例子

问：32位地址，页大小4KB，二级页表，页目录项占4字节，一个页目录能放多少个页目录项？

答：

- 页大小4KB = 4096字节
- 每个页目录项4字节
- (4096 / 4 = 1024) 个 → 正好 1024 = (2^{10}) → 所以页目录索引用10位，完美匹配。

课程总结

- 多级页表破除了页表存储的连续性要求，节省内存
- "填写页表该用虚拟地址还是物理地址"本质是鸡生蛋问题
- MIPS用硬件直接映射解决，内核有一块不需要页表就能访问
- X86用自映射解决，把页表自身映射到虚拟地址空间，内核用虚拟地址访问
- 单位换算、地址对齐这些基本功一定要熟练，实验考试都要用

这部分内容对你们做lab2、lab3非常重要，理解了自映射，填写页表就不会晕了。